

HR & PAYROLL MANAGEMENT PLATFORM

Sprint 3 High-Level Design

As-built design record — Waves 1, 2 & 3

Status	Delivered — all three waves shipped
Builds on	Sprint 1 MVP + Sprint 2
Generated	September 9, 2026

Scope: This document is the **as-built** record of Sprint 3, delivered as three independently-shipped waves: **Wave 1** (Recruitment expansion + Attendance/Ops hardening), **Wave 2** (People development), **Wave 3** (Finance + AI). All three waves are implemented, tested, and merged as of this document's revision.

Status	Delivered — Waves 1, 2, and 3 all shipped
Builds on	Sprint 1 MVP + Sprint 2 (geofenced attendance, expenses, notifications, recruitment/ATS)
Author	Generated with Claude Code, reviewed by knakhantryiaung-bot

1. Goals

Sprint 3 extends the platform along three fronts, phased so each wave shipped as an independently deployable, independently tested increment (matching how Sprint 2 sized its four-module delta):

- 1. Recruitment expansion & Attendance/Ops hardening (Wave 1).** Sprint 2's ATS was HR/recruiter-entered only and geofencing was circular-only; Wave 1 opens a public careers page with candidate self-service, adds polygon geofence zones, SMS/push notification channels, Google Calendar interview sync, and reimbursement outside the payroll cycle.
- 2. People development (Wave 2).** Asset management, Performance/KPI review cycles, and a lightweight Learning Management System (LMS) — all new employee-lifecycle surfaces built on Wave 1's precedents (adapter interfaces, force-scope resource access).
- 3. Finance & AI (Wave 3).** Bank disbursement file generation (extending Wave 1's manual reimbursement fields to an automated export) and a read-only AI HR assistant.

2. Roadmap & Phasing (as delivered)

Wave	Contents	Status
1	Careers page + candidate portal, polygon geofence, SMS/push channels, standalone reimbursement, Google Calendar integration	Delivered
2	Asset management, Performance/KPI reviews, Learning Management (LMS)	Delivered
3	Bank disbursement file export, AI HR agent	Delivered

Each wave's build order was chosen to keep every PR reviewable and every migration additive-only; see the companion Developer Handbook §1 for the exact sequencing used within each wave.

3. Cross-Cutting Decisions

These decisions were made once and then reused as precedent across later waves rather than re-litigated per feature:

- **Candidate identity is fully separate from internal auth (Wave 1).** Candidates get their own login, a distinct JWT (`CANDIDATE_JWT_SECRET` , own claim shape), and their own auth middleware (`requireCandidateAuth`). A candidate token must never satisfy `requireAuth` / `requireRole` — internal RBAC (`SUPER_ADMIN` / `HR_ADMIN` / `HIRING_MANAGER` / `EMPLOYEE`) is untouched.
- **External integrations are adapter-shaped**, mirroring `StorageAdapter` (`backend/src/common/storage/storageAdapter.ts`). SMS and push ship with a console-log stub provider (no vendor account needed to build/test/demo); swapping in Twilio/ `web-push` later is a provider-implementation change only. Google Calendar is the one integration with a real implementation from day one (product decision: Google-only, no Outlook/ICS).
- **"Force-scope" resource access, used everywhere an EMPLOYEE reads their own data.** List/get service functions accept `{ organizationId, requester: { userId, role }, query }`; when `requester.role` is not `HR_ADMIN` / `SUPER_ADMIN` , the function auto-restricts to the requester's own `Employee` record regardless of any `employeeId` filter in the query. Established for attendance/leave/expenses in Sprint 2, reused unchanged for assets, performance reviews, and course enrollments in Wave 2, and — in Wave 3 — deliberately **forced** for the AI assistant even when the real caller is HR_ADMIN/SUPER_ADMIN (§4.10), specifically to keep the assistant's data snapshot scoped to the asking employee no matter who's asking.
- **"Derived, not stored" status.** Where a status is fully determined by existing timestamp fields, no redundant status column is added. `PerformanceReview` 's PENDING/SELF_SUBMITTED/COMPLETED state is derived from `selfSubmittedAt` / `managerSubmittedAt` ; `CourseEnrollment` 's ENROLLED/COMPLETED state is derived from `completedAt` . This avoids a second copy of state that could drift from the timestamps that are the actual source of truth.
- **New geofence shape is additive, not a migration.** Existing circular zones keep working unmodified; polygon is a second shape option per zone.
- **Standalone reimbursement reuses the existing `ExpenseClaim` / `REIMBURSED` state** instead of a parallel model — manual disbursement and payroll-run disbursement are two paths to the same terminal state, mutually exclusive on a given claim.
- **Bank disbursement (Wave 3) is a generated payment file, not a live bank API call.** No real bank/aggregator integration exists for this deployment, so `BANK-01` produces an RFC 4180 CSV a finance operator uploads to their bank's own portal, rather than calling a vendor API that doesn't exist yet. This was an explicit product decision (see §4.9) made in place of the more ambitious "automate the existing `disbursementMethod` / `disbursementReference` fields from a real bank API" idea floated when Wave 1 shipped those columns.
- **AI HR agent (Wave 3) is single-shot context injection, not a tool-use loop.** Each chat turn builds a fresh, force-scoped data snapshot and hands it to the model as a system prompt; the model never calls tools or triggers writes. Deliberately narrow — see §4.10.

4. Feature Design

4.1 Public Careers Page & Candidate Self-Service Portal (Wave 1)

Requirement IDs: `CAREER-01` .. `CAREER-10` , extends `REC-01` .

Public surface (no auth). Each `Organization` has an optional unique `careersSlug` (set by HR Admin in org settings). Public careers URLs are slug-scoped — `GET /careers/:orgSlug/jobs` returns `OPEN` postings only (never salary bands or internal fields); `organizationId` is always resolved from the slug server-side, never supplied by the client. An org with no `careersSlug` has no public page. The one public write endpoints (`register` / `login`) share the existing login rate limiter.

Candidate portal (candidate auth). A new `CandidatePortalAccount` (1:1 with `Candidate`) holds `passwordHash` / `status` / `lastLoginAt` . `POST /careers/:orgSlug/register` creates the account (duplicate email within an org reuses the existing candidate record); `POST /careers/:orgSlug/login` issues a candidate JWT signed with `CANDIDATE_JWT_SECRET` . `requireCandidateAuth` re-reads account status on every request, same freshness rationale as `requireAuth` . Authenticated routes (`/candidate-portal/*`): `GET /me` , `POST /jobs/:jobId/apply` (resume upload via the existing `storageAdapter`), `GET /applications` , `GET /applications/:id` — the latter two are **field-filtered** (stage/schedule visible; interviewer identity, feedback, offer detail beyond status are not) via a dedicated serializer rather than an "if requester is a candidate" branch in `recruitment.service.ts` . Duplicate-open- application (REC-03) and single-active-offer (REC-05) checks apply unchanged — a portal-submitted application goes through the same service function as an HR-entered one.

Explicit non-goals: no email verification/password reset for candidates (matches the internal-user gap today); no CAPTCHA beyond rate limiting; no candidate-initiated slot self-scheduling.

4.2 Polygon Geofence Zones (Wave 1)

Requirement IDs: `GE0-10` .. `GE0-13` , continues Sprint 2's `GE0-01..09` .

`GeofenceZone` gained a `shape` enum (`CIRCLE` default, `POLYGON`); `lat` / `lng` / `radiusMeters` became nullable (`CIRCLE` -only) and a new `polygon Json?` holds an ordered `{lat, lng}` vertex list (3–50 points). Conditional-field validity is enforced in Zod + the service layer, not a DB constraint. Containment uses ray-casting point-in-polygon next to the existing haversine circular check; the attendance check-in path branches on `zone.shape` . Existing circular zones needed no migration beyond the new nullable columns with `shape` backfilled to `CIRCLE` . Frontend gained a polygon-drawing mode (`react-leaflet` + OpenStreetMap tiles, no API key) alongside the unchanged circle form.

4.3 SMS & Push Notification Channels (Wave 1)

Requirement IDs: `NOTIF-08` .. `NOTIF-13` , continues Sprint 2's `NOTIF-01..07` .

`NotificationPreference` gained `smsEnabled` / `pushEnabled` (in-app stays always-on). `User` gained `phoneNumber` (E.164). A new `PushSubscription` model holds standard Web Push subscription fields. Two new adapter interfaces (`backend/src/common/notifications/`) — `SmsProvider` , `PushProvider` — ship with console-log stub implementations; a real vendor is a provider swap only. `notification.queue.ts` 's per-recipient loop fans out to SMS/push after the existing email-stub call, same transaction and idempotency guarantee as Sprint 2's NOTIF-03; a channel-specific send failure is caught and logged per-channel, never failing the whole event. `notification.templates.ts` gained a per-event `smsBody` (≤160 chars, link/ reference only — same no-sensitive-data rule as NOTIF-04).

4.4 Standalone Reimbursement (Wave 1)

Requirement IDs: EXP-09 .. EXP-11 , continues Sprint 2's EXP-01..08 .

ExpenseClaim gained four nullable columns: disbursementMethod (BANK_TRANSFER / CHEQUE / CASH / OTHER), disbursementReference , disbursedAt , disbursedBy . POST /expenses/:id/reimburse (HR Admin/Super Admin) moves an APPROVED claim directly to REIMBURSED , independent of any payroll run. The existing payroll-cycle reimbursement path is untouched; the two paths are mutually exclusive on a given claim, enforced in the service. This is the shape Wave 3's bank disbursement export plugs into (§4.9) — disbursementMethod / disbursementReference are HR-admin-typed strings on this path, machine-generated CSV rows on that one, same columns either way.

4.5 Google Calendar Integration for Interviews (Wave 1)

Requirement IDs: CAL-01 .. CAL-07 .

Org-level connection (one HR Admin authorizes a single Google account/calendar per org, not per-interviewer). New CalendarIntegration model holds AES-256-GCM-encrypted OAuth tokens (CALENDAR_TOKEN_ENC_KEY) — the first at-rest secret encryption in this codebase (passwords use one-way bcrypt hashing, which doesn't apply here since the raw token must be recoverable to call Google's API). Interview.interviewerNames (free-text) was replaced by structured interviewers Json ({name, email}[]) with a one-time backfill migration (comma-split into [{name, email: null}]), plus a new calendarEventId for update/cancel. CalendarProvider interface (createEvent / updateEvent / cancelEvent), GoogleCalendarProvider the sole implementation. Interview create/reschedule/cancel calls the provider synchronously in the same request — a failed calendar call surfaces as a warning but the interview record is still saved. Without a connected integration, scheduling degrades to Sprint 2's behavior (structured interviewer list, no calendar event) rather than blocking.

4.6 Asset Management (Wave 2)

Requirement IDs: ASSET-01 .. ASSET-08 .

Scope decision (confirmed with product owner): IT/company equipment only, assign-and-return workflow — no depreciation/finance tracking, no barcode scanning. Asset (assetTag unique per org, category enum LAPTOP / MONITOR / PHONE / PERIPHERAL / FURNITURE / OTHER , status enum AVAILABLE / ASSIGNED / IN_REPAIR / RETIRED , currentEmployeeId) plus AssetAssignment , an append-only history row per assign/return cycle (assignedAt / assignedBy , returnedAt / returnedBy). Asset.status is only ever set to ASSIGNED via the assign action, never a direct PATCH — updateAsset rejects a direct write to ASSIGNED with a conflict error (ASSET_CURRENTLY_ASSIGNED), preventing status / currentEmployeeId / AssetAssignment from ever going out of sync. POST /assets/:id/assign requires the asset to be AVAILABLE ; POST /assets/:id/return clears currentEmployeeId back to AVAILABLE and closes the open AssetAssignment row. GET /assets is HR-unrestricted, force-scoped to the caller's own currentEmployeeId for anyone else (§3).

4.7 Performance / KPI Reviews (Wave 2)

Requirement IDs: PERF-01 .. PERF-07 .

Scope decision: periodic review cycles (self-assessment + manager assessment), not continuous/OKR-style tracking. `Employee` gained a self-relation `managerId`. `PerformanceReviewCycle` (`DRAFT` → `OPEN` → `CLOSED`, matching the payroll-run lifecycle shape at a smaller scale) and `PerformanceReview` (one row per cycle × employee, created in bulk for every then- `ACTIVE` employee when a cycle opens). No stored status column — `PENDING/SELF_SUBMITTED/COMPLETED` is derived from `selfSubmittedAt` / `managerSubmittedAt` (§3). `PATCH /performance/reviews/:id/self` and `.../manager` are separate actions with separate rating/comment fields; the manager section can be submitted by the employee's `managerId` or, if unset, an HR Admin/Super Admin standing in — the actual submitter is recorded in `managerUserId` since it can differ from `Employee.managerId`. `goals` is free-text for the period — no separate Goal entity with its own lifecycle in Wave 2 scope. List/get access is resource-scoped to "own review, or a direct report's" (`employee.managerId === requester's employee`), one step narrower than the plain self-only force-scope used elsewhere, since a manager needs to read (and complete) their reports' reviews.

4.8 Learning Management (LMS) (Wave 2)

Requirement IDs: LMS-01 .. LMS-07 .

Scope decision: link-out + self-reported completion, not in-app content hosting or SCORM. `Course` (`externalUrl`, `category`, `mandatory` flag, `status` `ACTIVE` / `ARCHIVED` for retiring outdated courses without deleting them) and `CourseEnrollment` (one row per course × employee; `completedAt` is the entire completion model — no manager sign-off step). `POST /courses/:id/enroll` is self-service; `POST /courses/:id/enroll-all-active` (HR-only) bulk-enrolls every active employee, e.g. for a mandatory compliance course. `POST /courses/:id/complete` is self-reported by the enrolled employee. `GET /courses/enrollments` is force-scoped the same way as assets (§3): HR sees every enrollment in the org, anyone else sees only their own.

4.9 Bank Disbursement File Export (Wave 3)

Requirement ID: BANK-01 .

Product decision (§3): generate a payment file rather than integrate a real bank API, since no bank/aggregator relationship exists for this deployment. `Employee` gained `bankName` / `bankAccountName` / `bankAccountNumber` (all nullable — blank until HR fills them in via the employee detail screen). Two new read-only, HR-only export endpoints, both streaming `text/csv` with a `Content-Disposition: attachment` header:

- `GET /payroll/runs/:id/disbursement-file` — one row per `PayrollItem` in a payroll run whose status is `APPROVED` or `PAID` (a run must have reached that stage before its payout file is generated); rejects with `PAYROLL_RUN_NOT_APPROVED` otherwise.
- `GET /expenses/disbursement-file` — one row per `APPROVED` expense claim not yet attached to a `PayrollItem` (i.e. eligible for standalone disbursement, §4.4); read-only export, does **not** mark anything `REIMBURSED` — HR still confirms the actual payout via `POST /expenses/:id/reimburse`

after uploading the file to their bank.

The CSV writer (`common/csv/buildCsv.ts`) is a small hand-rolled RFC 4180 implementation (quote/escape every cell, `\r\n` line endings) rather than a new dependency, since the shape needed — a header row plus flat scalar rows — doesn't warrant one.

4.10 AI HR Agent (Wave 3)

Requirement IDs: `AI-01` .. `AI-05` .

Scope decision (§3): read-only Q&A over the **asking employee's own** HR data, using the Claude API (`@anthropic-ai/sdk`), with no write actions and no access to any other employee's data — the narrowest version of the idea sketched when Wave 1 shipped, chosen specifically because a write-capable HR agent is high blast-radius and better earned incrementally than started with.

Design: single-shot context injection, not a tool-use loop. Each `POST /ai/chat` call:

1. Resolves the calling user's own `Employee` record.
2. Builds a snapshot by calling the *existing* self-scoped list functions across every relevant module (leave balances, recent attendance, recent payslips, expense claims, assigned assets, performance reviews, course enrollments, basic profile) — `Promise.all` 'd for latency, and each call passes a **synthetic** `{ userId, role: "EMPLOYEE" }` requester regardless of the caller's actual role. This is the force-scope pattern from §3, deliberately applied even when the real caller is `HR_ADMIN` / `SUPER_ADMIN` : an HR Admin using the assistant must never get org-wide data back through it just because their real role would normally unlock that in `listLeaveBalances` / `listAttendance` /etc.
3. Serializes the snapshot into a system prompt with explicit instructions: answer only from the given data, never fabricate numbers, disclose no access to any other employee's data, and take no actions (the assistant cannot submit or approve anything — it can only answer questions).
4. Calls the Claude Messages API once (`env.anthropicModel` , default `claude-sonnet-5`) with that system prompt plus the client-supplied conversation history and the new message.

No server-side chat history. The client resends prior turns (capped at 20) with every request; nothing is persisted. This keeps the feature stateless and avoids adding a new place sensitive HR data would be stored at rest beyond what the source-of-truth tables already hold.

Configuration is opt-in, same as Calendar/Bank in earlier waves. Without `ANTHROPIC_API_KEY` set, `POST /ai/chat` returns a clear `AI_ASSISTANT_NOT_CONFIGURED` error rather than the app failing to start.

Rate limiting. A dedicated per-user limiter (`aiChatRateLimiter` , keyed on `req.auth.userId` rather than IP) protects against runaway usage against a paid, per-token external API — the first endpoint in this codebase with that cost profile.

5. Non-Functional Requirements

- **Tenancy isolation:** every public/candidate endpoint resolves `organizationId` server-side (via slug or candidate JWT), never from client input.

- **Secret isolation:** candidate JWTs use a distinct secret from internal JWTs by design, not just a claim check.
- **Resource-scope isolation:** every Wave 2/3 self-service read (assets, reviews, enrollments, and the AI snapshot) uses the force-scope pattern (§3) so an EMPLOYEE requester — real or synthetic — can never read another employee's row through it.
- **Backward compatibility:** no Sprint 3 change required migrating or invalidating existing Sprint 1/2 data; every schema change is additive except the `interviewerNames` → `interviewers` rename, which shipped with a lossless backfill.
- **Testing:** every new capability follows the existing `*.test.ts` (vitest + supertest) convention, module-scoped, hitting a real Postgres test database.
- **Graceful degradation:** every external integration (Calendar, SMS/push, AI) is opt-in — unset credentials produce a clear configuration error on the specific endpoint that needs them, never an app-wide startup failure.

6. Explicit Out of Scope (all of Sprint 3)

- Outlook/Microsoft 365 calendar support, ICS fallback.
 - Real SMS/push vendor wiring (ships as console-log stubs behind the adapter).
 - Candidate email verification / password reset.
 - Per-interviewer (vs. org-level) calendar connections.
 - Asset depreciation/finance tracking, barcode/RFID scanning.
 - Continuous/OKR-style performance tracking; a separate Goal entity with its own lifecycle.
 - In-app LMS content hosting, SCORM packages, quizzes/certifications.
 - A live bank/payment-aggregator API integration (ships as a generated payment file).
 - Any AI agent write action (submitting requests, approving anything) or access to another employee's data.
-

Appendix A — Requirement ID Index

Series	Area	Wave
CAREER-01..10	Public careers page + candidate portal	1
GEO-10..13	Polygon geofence zones	1
NOTIF-08..13	SMS/push notification channels	1
EXP-09..11	Standalone reimbursement	1
CAL-01..07	Google Calendar integration	1
ASSET-01..08	Asset management	2
PERF-01..07	Performance/KPI reviews	2
LMS-01..07	Learning management	2
BANK-01	Bank disbursement file export	3
AI-01..05	AI HR agent	3